System Design and Implementation

We propose a modular safety-filter architecture (an “Adaptive ShieldNN”) that augments any nominal controller with a learned safety margin $\delta(x)$ and a closed-form override $\lambda(x)$. In our design, the *state* $x = [x, y, \psi, v]$ of the kinematic bicycle model is fed both to a policy network that outputs an adaptive margin $\delta(x)$ and to a barrier-function module. The barrier computes the safety function $h(x)$ (e.g. a radial distance-based barrier) and its Lie derivative $\dot{h}(x)$ under nominal control. We then enforce the CBF constraint

$\dot{h}(x) + \alpha(h(x)) \geq \delta(x)$,

solving analytically for a corrective control $\lambda(x)$ when necessary. The override control is applied as

u_safe = u_nominal + $\lambda(x)$,

similar to ShieldNN [1], but with a state-dependent safety offset $\delta(x)$ instead of a fixed threshold.

The overall system flow is: at each time step, the vehicle state enters the **safety filter module** where (1) the adaptive margin $\delta(x)$ is computed by a learned network, (2) the CBF constraint is evaluated, and (3) the corrective term $\lambda(x)$ is calculated in closed form if needed. The sum `u_safe = u_nominal +` is then applied to the vehicle. Importantly, we **certify** $\delta(x)$ by enforcing $\delta(x) \geq \delta_min(x)$ at each state, where $\delta_min(x)$ is a worst-case bound derived from symbolic derivative certificates. If the learned δ is too small, we clip it to $\delta_min(x)$ (or switch to a fallback control) to guarantee safety.

Key modules in implementation:

- **KBM Environment:** Implements Kinematic Bicycle dynamics (state update for $[x, y, \psi, v]$ given control $[\delta_f, a]$).
- **Barrier Function:** Computes $h(x)$ (e.g. `h = r_bar - sqrt(x^2+y^2)`) and its derivative $\dot{h}(x,u)$ under control u. We enforce $\dot{h}+\alpha(h)\geq\delta$. (Here $\alpha(h)$ is a class-□ function, e.g. $\alpha(h) = \gamma \cdot h$.)
- **Adaptive Margin Network (δ-net):** A PyTorch neural network taking state x (and optionally additional context) and outputting $\delta(x)\geq 0$. This replaces fixed “radius offsets” from ShieldNN [1].
- **Closed-form Override (λ):** An analytical computation adjusting the steering (and/or acceleration) just enough to satisfy the CBF inequality. For example, if $\dot{h} + \alpha(h) < \delta$, solve for λ so that $(\dot{h} + \partial\dot{h}/\partial u \cdot \lambda) + \alpha(h) = \delta$, applying only the minimal corrective steering change (as in ShieldNN).
- **Certified Fallback (δ_min):** Pre-computed numeric bounds $\delta_min(x)$ (from `deriv*_certs.p`) ensure $\delta(x)$ never falls below a safe threshold. If $\delta(x) < \delta_min(x)$, we set $\delta=\delta_min$ or switch to a backup policy (e.g. maximum safe steering).
- **Reward Shaping:** The RL reward balances task performance (e.g. forward velocity or track following) with safety and smoothness. We penalize large δ (encouraging tight margins), penalize $\|\Delta u\|^2$ to enforce smooth control, and heavily penalize any violation of $\dot{h}+\alpha(h)<0$.

Below is a block-diagram of the system (conceptual):

- **State x → Adaptive Margin δ-net** → $\delta(x)$

- **State x → Barrier h, h□ →** *Constraint* $\dot{h}+\alpha(h) \geq \delta(x)$ **→ Compute λ**
- **Nominal Control u_nominal → Apply λ → u_safe → KBM Env** (next state)

Note: For notational simplicity we combine λ calculation and addition into a single "Safe Controller" block in the code.

Figure 1: *Block diagram of the Adaptive ShieldNN safety filter.* The learned δ(x) is plugged into the CBF constraint and solved by a closed-form override λ(x). A certified bound δ_min(x) prevents unsafe underestimation. (Unlike generic CBF-QP methods, we compute λ analytically [1], ensuring real-time speed.)

The *proposed implementation order* is:

1. **Barrier & Dynamics Module:** Implement the KBM dynamics and barrier function h(x), α(h). (Utilize Yasser's MATLAB derivations if needed.)
2. **Safe Controller (`safe_controller.p`):** Write functions that compute h, $\dot{h}$, and solve for λ. Begin with a simple radial barrier (e.g. distance to obstacle) and linear approximation for λ as shown below. Ensure this filter works for a nominal controller (e.g., tries to track a goal).
3. **Certified Margin Loading (`certify_delta_margin.p`):** Load the δ_min certificates (from `deriv*_certs.p`) and implement logic to clip δ(x).
4. **Reward Shaping (`reward_shaping.py`):** Define the RL reward combining task, margin tightness, smoothness, and penalties for violation.
5. **Adaptive Margin Network & Training Loop (`train_adaptive_margin.p`):** Set up the RL framework. Define a PyTorch policy network δ(x) (e.g. a small MLP). Integrate with a Gym-style KBM environment so that each step:
6. Query δ = policy(x),
7. Apply `δ = certify_delta(δ,x)`,
8. Compute safe control `u_safe = SafeController(x, u_nominal, δ)`,
9. Step the environment with `u_safe`.
   Use PPO or A2C to train the δ-network.
10. **Baseline Controllers:** Implement the fixed-margin ShieldNN (for comparison) and a QP-CBF filter.
11. **Visualization (`visualize_results.p`):** Plot trajectories, δ(x) over time, λ(x), safety violations, and metrics (Figure examples given below).

At each step, verify consistency with known results from ShieldNN: for example, ensure the safe set nearly matches that reported in [1] when δ(x) is fixed.

Advice and Warnings:
- *Real-time override:* Because the CBF override λ is derived in closed form, it can be computed quickly at runtime. However, ensure numerical stability (e.g. add small epsilon in divisions when computing λ).
- *Network expressive power:* The δ(x)-network should be expressive enough to represent varying margins. A few hidden layers (like the structure in Yasser's exported networks [2]) is a good starting point.
- *Reward tuning:* Balancing safety vs performance is delicate. The fixed threshold δ_min is our hard safety guarantee; rely on rewards to push δ(x) down towards δ_min but never below.
- *Comparisons:* We must compare against the original ShieldNN results [1]. Use similar metrics (e.g. RL episode success rate with/without shield).
- *Logging:* During training and evaluation, log safety violations, margin values, and control smoothness (e.g. $\|\Delta u\|^2$). Visualize δ(x) on the state-space (or over time) to confirm it adapts to context.
- *Testing corner cases:* Verify behavior on tight turns or high speeds. If δ_min clipping triggers often, the learning reward may need adjusting.

Below are implementation-ready code snippets for each component. Each file is a self-contained Python module. (All examples use PyTorch for learning and numpy for dynamics; feel free to adapt to TensorFlow if preferred. But PyTorch is recommended based on user preference.)

```
# safe_controller.py
import numpy as np

L = 2.0    # wheelbase length [m] (example
value) r_bar = 5.0 # Safe distance threshold [m]
alpha_gain = 1.0  # h_dot + gamma*h >= δ, here gamma=alpha_gain

def barrier(x):
    """
    Radial safety barrier h(x) = r_bar -
    distance_to_origin. Returns h >= 0 if inside safe
    region (r < r_bar).
    """
    px, py = x[0], x[1]
    r = np.sqrt(px*px + py*py)
    return r_bar - r

def barrier_derivative(x, u):
    """
    Compute h_dot = d/dt (h(x)) under control
    u. For h = r_bar - sqrt(x^2+y^2), we
    have:
        r_dot = (x*vx + y*vy) / r
    where vx = v*cos(ψ), vy =
    v*sin(ψ).
    We ignore higher-order steering effects for
    simplicity. """
    px, py, psi, v = x
    delta_f, a = u
    r = np.sqrt(px*px + py*py) + 1e-6
    vx = v * np.cos(psi)
    vy = v * np.sin(psi)
    r_dot = (px*vx + py*vy) / r
    # Since h = r_bar - r, h_dot = -r_dot.
    return -r_dot

def compute_lambda(x, u_nom,
    delta_margin): """
    Compute minimal corrective steering (lambda) so that
    h_dot(x, u_nom + [lambda, 0]) + alpha*h(x) >=
    delta_margin.
    We only adjust the steering (first control), keeping acceleration
    fixed. This uses a simple linear approximation.
    """
    h = barrier(x)
    h_dot_nom = barrier_derivative(x,
    u_nom) # current safety value
    safety_val = h_dot_nom + alpha_gain *
    h if safety_val >= delta_margin:
        # Already safe, no correction needed
```

```
        return np.zeros_like(u_nom)
    # Otherwise compute required delta_lambda

# Simple proportional law: (h_dot + k*lambda) + alpha*h = δ -> solve for
lambda
    # Here we assume steering change maps linearly to h_dot; we pick a gain
factor k.
    k = 1.0
    lambda_val = (delta_margin - safety_val) *
    k lam = np.array([lambda_val, 0.0])
    return lam

def safe_control(x, u_nominal,
    delta_margin): """
    Apply the safety filter: compute override λ and produce safe
    control. Returns (u_safe, lam).
    """
    u_nominal = np.array(u_nominal, dtype=float)
    lam = compute_lambda(x, u_nominal, delta_margin)
    u_safe = u_nominal + lam
    # Clip controls to physical limits
    u_safe[0] = np.clip(u_safe[0], -0.5, 0.5) # steering limit (rad)
    u_safe[1] = np.clip(u_safe[1], -1.0, 1.0) # acceleration limit
    (m/s^2) return u_safe, lam
```

```
# reward_shaping.py
import numpy as np

def compute_reward(state, state_next, u, u_prev, delta_margin,
    violated): """
    Total reward = task_reward + margin_penalty + smoothness_penalty +
violation_penalty.

    - task_reward: e.g. forward velocity (towards goal).
    - margin_penalty: penalize large δ (encourage minimal safety slack).
    - smoothness_penalty: penalize large control jumps (u - u_prev).
    - violation_penalty: heavy penalty if safety inequality
    violated. """
    px, py, psi, v = state
    # Example task: reward forward motion (x-direction
    velocity) vx = v * np.cos(psi)
    task_reward = vx

    # Penalize large safety margin
    margin_penalty = -10.0 * (delta_margin**2)

    # Control smoothness penalty
    du = np.array(u) - np.array(u_prev)
    smoothness_penalty = - np.linalg.norm(du)**2
```

```
    # Safety violation penalty
    violation_penalty = -100.0 if violated else 0.0

    return task_reward + margin_penalty + smoothness_penalty +
violation_penalty
```

```
# certify_delta_margin.py
import pickle

# Load certified δ_min data (example: from deriv*_certs.p
files) def load_certified_data(path):
    with open(path, 'rb') as f:
        data = pickle.load(f)
    return data

# Example: a global variable holding certified bounds
delta_certs = None

def certify_delta(delta_pred, state):
    """
    Ensure the learned margin δ >= δ_min(state). If not, clip to
    δ_min. In practice, δ_min might depend on state (e.g., radial
    coordinate).
    Here we use a constant lower bound from loaded certificates for
simplicity.
    """
    global delta_certs
    if delta_certs is None:
        delta_certs = load_certified_data('deriv1_certs.p')       # path to
    data # For demo, assume delta_certs contains a key 'min_margin'
    delta_min = delta_certs.get('min_margin', 0.1)
    return max(delta_pred, delta_min)
```

```
# visualize_results.py
import matplotlib.pyplot as plt
import numpy as np

def plot_trajectory(states,
    label='Trajectory'): xs = [s[0] for s in
    states]
    ys = [s[1] for s in
    states] plt.plot(xs, ys,
    label=label) plt.xlabel('x
    position') plt.ylabel('y
    position') plt.legend()
    plt.grid(True)

def plot_margin(delta_history):
    plt.figure()
    plt.plot(delta_history, label='δ(x) over time')
    plt.xlabel('Timestep')
```

```
    plt.ylabel('Safety margin δ')
    plt.title('Adaptive Safety Margin')
    plt.legend()
    plt.show()

def plot_controls(u_history):
    u_hist = np.array(u_history)
    plt.figure()
    plt.plot(u_hist[:,0], label='Steering δf')
    plt.plot(u_hist[:,1], label='Acceleration a')
    plt.title('Control Inputs Over Time')
    plt.xlabel('Timestep')
    plt.ylabel('Control Value')
    plt.legend()
    plt.grid(True)
    plt.show()

def plot_lambda(lambda_history):
    lamb = np.array(lambda_history)
    plt.figure()
    plt.plot(lamb[:,0], label='Steering override λ')
    plt.title('Safety Override λ(x)')
    plt.xlabel('Timestep')
    plt.ylabel('λ value')
    plt.legend()
    plt.grid(True)
    plt.show()
```

```
# train_adaptive_margin.py
import gym
import torch
import torch.nn as nn
import torch.optim as
optim
from safe_controller import safe_control
from reward_shaping import
compute_reward
from certify_delta_margin import certify_delta

# Simple Gym-like KBM environment (outline)
class KBMEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.dt = 0.1
        # State: [x, y, psi, v]
        self.state = np.zeros(4)
    def reset(self):
        # Initialize state, e.g., near track start
        self.state = np.array([0.0, -2.0, 0.0, 0.0])
        return self.state
    def step(self, u):
        x, y, psi, v = self.state
```

```
        delta_f, a = u
        # Kinematic bicycle update (discrete
        Euler) x_dot = v * np.cos(psi)
        y_dot = v * np.sin(psi)
        psi_dot = v * np.tan(delta_f) / 2.0    # L=2.0
        v_dot = a
        # Update state
        x += x_dot *
        self.dt y += y_dot *
        self.dt
        psi += psi_dot *
        self.dt v += v_dot *
        self.dt
        self.state = np.array([x, y, psi, v])
        # Example done condition: out of bounds or too many
        steps done = (abs(x) > 10 or abs(y) > 10)
        # Placeholder reward (actual computed externally)
        return self.state, 0.0, done, {}

# Policy network producing
δ(x) class
MarginNet(nn.Module):
    def _init_(self):
        super()._init_()
        self.net = nn.Sequential(
            nn.Linear(4, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.Softplus()    # ensures output ≥ 0
        )
    def forward(self, x):
        return self.net(x)

env = KBMEnv()
policy = MarginNet()
optimizer = optim.Adam(policy.parameters(), lr=1e-3)

# PPO or A2C training loop outline
for episode in range(1000):
    state = env.reset()
    u_prev = np.array([0.0,
    0.0]) done = False
    log_probs =
    [] rewards =
    [] states = []
    while not
    done:
        state_tensor = torch.tensor(state, dtype=torch.float32)
        delta = policy(state_tensor).item()
        # Certify margin
        delta = certify_delta(delta, state)
        # Define some nominal control (e.g., try to go towards y=0
        line) # This could be a simple P-controller on lateral error
```

```
        nominal_u = np.array([0.0, 0.1])   # e.g., slight forward
        accel # Compute safe control
        u_safe, lam = safe_control(state, nominal_u, delta)
        next_state, _, done, _ = env.step(u_safe)
        # Safety violation flag
        h = barrier(state)
        h_dot = barrier_derivative(state, nominal_u)
        safe_violation = (h_dot + alpha_gain*h <
        delta) # Compute reward
        r = compute_reward(state, next_state, u_safe, u_prev, delta,
safe_violation)
        rewards.append(r)
        # Store log prob (for policy gradient)
        states.append(state_tensor)
        # (For simplicity, we skip an actual log-prob computation here.)
        state = next_state
        u_prev = u_safe

    # After episode: compute returns and update policy (simplified)
    R = 0
    returns = []
    for r in rewards[::-
        1]: R = r + 0.99 *
        R
        returns.insert(0, R)
    returns =
    torch.tensor(returns) states =
    torch.stack(states)
    # Example loss: (return * δ) to push δ down/up
    loss = -torch.mean(returns * policy(states).squeeze())
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print("Training completed.")
```

Proposed Implementation Order: 1. **Implement and test the KBM environment** in Gym style. Verify bare dynamics (e.g. simple forward acceleration yields correct motion).

2. **Implement the barrier function and override (safe_controller.py)**. Verify on sample states that the override λ prevents entering unsafe region.

3. **Load certified bounds (certify_delta_margin.py)** and ensure δ is clipped to δ_min(x).

4. **Define and test the reward (reward_shaping.py)** by hand on some trajectories.

5. **Set up the RL loop (train_adaptive_margin.py)**. Start with random δ output to verify integration. Use PPO/A2C libraries if available, or the above simple policy gradient.

6. **Experiment with baseline controllers**: ShieldNN fixed margins, QP-CBF filter.

7. **Run training and evaluation**, logging violations, δ-values, and control smoothness.

8. **Visualize results (visualize_results.py)**, comparing trajectories with/without adaptive margins.

Paper Structure Outline

Title: *Adaptive ShieldNN: Safe Closed-Form Control with Learned Margins and Smoothness Penalties*

Introduction

Introduce the challenge of safe RL in robotics: RL policies can explore unsafe actions, so safety filters like Control Barrier Functions (CBFs) are used [3] [1]. Describe ShieldNN [1]: it provides a closed-form safe controller for the kinematic bicycle model with obstacle avoidance. Emphasize our contribution: instead of fixed safety margins, we *learn* an adaptive margin δ(x) via RL, and add a smoothness penalty for control stability. Outline: we incorporate a certified fallback δ_min(x) from derivative bounds, yielding provable safety with improved performance. Cite ShieldNN and safe-RL surveys [3] [1].

Related Work

- **Safe RL and CBFs:** CBFs are widely used to enforce safety in control and RL [3]. Prior work includes RL-with-CBF layers and shield methods [3]. However, designing CBFs manually is challenging [3].
- **ShieldNN:** Review Ferlez *et al.*'s ShieldNN [1], which provided the first provably-safe NN-based filter for the bicycle model, giving a closed-form safe action set (without optimization).
- **Smoothness in RL:** Recent works target action smoothness. *LipsNet* [4] constrains network Lipschitz constants to reduce action fluctuation; *SmODE* [5] uses neural-ODE layers as low-pass filters. These inform our smoothness penalty (we penalize Δu^2 in the reward, akin to Lipschitz regularization [4] [5]).
- **Adaptive Safety Margins:** Some CBF methods learn state-dependent margins or relaxations, but typically lack closed-form overrides. Our adaptive margin δ(x) is unique in combining learning with analytical safety.

Preliminaries

Control Barrier Functions (CBFs): Define h(x) (e.g. distance-based barrier) and the forward invariant condition $\dot{h}(x,u)+\alpha(h) \geq 0$ for safety [3].
Kinematic Bicycle Model: Equations for state [x,y,ψ,v] and control [δ_f,a]. Include steering and acceleration constraints.
ShieldNN Overview: Summarize ShieldNN's approach: a barrier and its Lie derivatives yield a constraint on u, solved in closed-form for λ [1]. The safe control set is derived offline (via neural nets approximating safe region boundaries [2]) and applied online as u_safe = u_nominal+λ. Our method extends this by making the margin δ state-dependent.

Proposed Method

- **Adaptive Margin δ(x):** Introduce a neural network that outputs δ≥0 for each state. This replaces the fixed "radius offset" used in ShieldNN. We train δ(x) via RL so it is tight (small) in easy situations and larger near complex obstacles.
- **Modified Safety Constraint:** The CBF condition becomes $\dot{h}(x,u) + \alpha(h(x)) \geq \delta(x)$. Geometrically, this allows the boundary of the safe set to shrink or expand per state.
- **Closed-form Override λ(x):** Detail how we compute λ: solve $\dot{h}(x, u_{nom}+\lambda) + \alpha(h) = \delta(x)$ for λ, applying only to the steering input. This yields a minimal correction to ensure safety, analogous to [1].
- **Smoothness Regularization:** We add a term $-\|\Delta u\|^2$ to the reward to discourage jerky actions between time-steps, inspired by LipsNet/SmODE ideas [4] [5]. This biases the learning towards smoother δ(x) that require gentler λ.
- **Certified Fallback δ_min(x):** Using Yasser's derivative certificates (in `deriv*_certs.p`), we compute a state-dependent lower bound δ_min(x) such that any δ < δ_min(x) would risk

violation. At runtime, we enforce $\delta(x) \geq \delta_min(x)$ (clipping or activating a backup policy). This ensures formal safety despite learning (similar to robust CBF ideas[3]).

Figures: For example, include a plot of the learned $\delta(x)$ over a state-space slice, showing adaptivity. Another figure could compare trajectories with adaptive vs fixed δ.

Training Pipeline

Describe the RL setup: observations = x, action = $\delta(x)$. Environment steps by applying u_safe.

- **Reward:** Combine task reward (e.g. forward progress or goal distance), margin penalty ($-10\cdot\delta^2$), smoothness penalty ($-\|\Delta u\|^2$), and a large negative reward on any safety violation.
- **Algorithm:** We use PPO (or A2C) in PyTorch, training δ-net with standard policy-gradient. (Hyperparameters: learning rate, γ, etc.) We follow ShieldNN's idea of "shielding" during training, so unsafe actions never corrupt the dynamics.
- **Nominal Controller:** Use a simple baseline controller for the task (e.g. a PID lane-keeper or waypoint follower). The RL only learns $\delta(x)$ on top of this.
- Optionally show a flowchart of training (state → δ-net → safe filter → reward/backprop).

Experiments

- **Setup:** Test in simulation (e.g. static obstacles around track center). Possibly use an F1TENTH racing simulator for realism. Vary scenarios: tight turns, obstacle proximities, dynamic obstacles.
- **Metrics:** Report safety violations (percentage of time $h<0$), task success (lap completion or track-center deviation), control smoothness (average $\|\Delta u\|$), and average δ.
- **Baselines:** (1) *ShieldNN-fixed*: original ShieldNN with constant safety margin (using Yasser's β-network)[1]. (2) *QP-CBF filter*: a quadratic-program-based CBF (slower). (3) *No filter*: the nominal controller alone.
- **Ablations:** Remove smoothness penalty (to show action jitter), remove δ_min constraint (to see how learning violates safety), and remove the override (to see necessity of λ).
- **Results:** Provide tables/plots:
- **Quantitative:** e.g. safety violations vs. method, task reward vs. method.
- **Plots:** Trajectory plots showing safer paths. Heatmaps of $\delta(x)$ over states. $\lambda(x)$ profiles showing how override kicks in.
- For example, show $\delta(x)$ along a difficult turn (it should increase) versus easy straight (small).
- We will compare with ShieldNN's reported rates: *"ShieldNN dramatically increased completion rate of RL episodes"*[1], and demonstrate that our Adaptive ShieldNN further improves completion with tighter safety margins.

Discussion and Future Work

Discuss findings: e.g. adaptive δ can reduce conservativeness (smaller margins away from obstacles) while maintaining safety. Smoothness penalty yields visibly smoother controls at minor cost in optimality. The certified fallback adds robustness but may slow learning if too strict.

Future directions: incorporate velocity into the barrier (a dynamic, velocity-aware $h(x)$), use Graph Neural Networks to model multi-obstacle $\delta(x)$, or employ curriculum learning to slowly tighten safety constraints. For sim-to-real, plan transfer to CARLA or a real platform (e.g. F1TENTH car) using our certified safety fallback.

Appendix

Include link to code repository, derivation details for the closed-form λ, and network architecture diagrams. Provide additional plots (e.g. δ(x) landscapes) and training curves.

References: In the final paper, we will cite Ferlez *et al.*'s ShieldNN[1], LipsNet[4][6], SmODE[5], and recent safe-RL surveys[3], among others, to situate our contribution. (For brevity, citation keys like [4], [12], [24], [30] above denote these sources.)

---

1 [2006.09564] ShieldNN: A Provably Safe NN Filter for Unsafe NN Controllers
https://arxiv.org/abs/2006.09564

2 model.py
file://file-MRu7eG1VB3QEHhFBznHMEF

3 Learning Control Barrier Functions and their application in Reinforcement Learning: A Survey
https://arxiv.org/html/2404.16879v1

4 LipsNet: A Smooth and Robust Neural Network with Adaptive Lipschitz Constant for High Accuracy Optimal Control
https://proceedings.mlr.press/v202/song23b.html

5 ODE-based Smoothing Neural Network for Reinforcement Learning Tasks | OpenReview
https://openreview.net/forum?id=S5Yo6w3n3f

6 LipsNet++: Unifying Filter and Controller into a Policy Network
https://openreview.net/pdf/cc6e495e72774ec7574697787def1b2e48378771.pdf